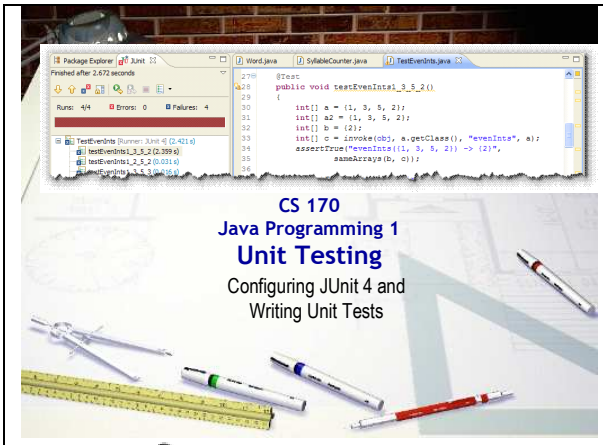


Slide 1

CS 170
Java Programming 1
Unit Testing
Duration: 00:00:50
Advance mode: Auto

Notes:

Hi there. Welcome to the CS 170, Java Programming 1 lecture on Unit Testing.

Unit testing isn't anything new for you, of course. Your very first programs were graded using Web-Cat and the reference tests that I wrote, and, starting in Chapter 2 you began writing and running your own unit tests using BlueJ's test recording feature.

In this lecture you'll learn how to write unit tests using Eclipse. We'll continue to use the JUnit testing framework, just like we did with BlueJ. With Eclipse, though, we'll use the more modern version of JUnit, JUnit 4, and instead of using BlueJ's test recorder, you'll learn how to write your unit tests by calling the methods in the JUnit Assert class.

Slide 2

Testing: the Big Ideas
Duration: 00:03:06
Advance mode: Auto

Notes:

Both testing and debugging are really two sides of the same coin; both are designed to ensure that your program runs correctly. Testing is designed to expose errors, while debugging is used to locate and fix the errors that are discovered through testing. Of course once you've fixed an error, you need to make sure you test it to show that it does what you expect.

One person who has been very influential in the area of testing is Edsgar Dijstra, the Dutch professor famous for his "Goto considered harmful" paper. Dijkstra is responsible for two really big ideas when it comes to testing:

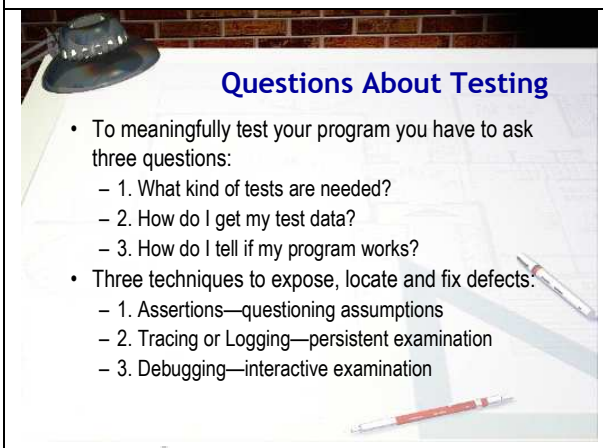
- First is the idea that testing doesn't prove that your program is correct. Thorough testing can show the presence of bugs, but it can never prove that all the bugs in your code have been found. Dijkstra was a proponent of software verification and the discipline known as formal methods and he wanted to make a distinction verified code—that is, code that is mathematically proven to be correct—and code that was well tested. In fact, though, the promise of formal verification has not proved to be as useful as Dijkstra first hoped, both because of the difficulty and the cost.

Of course, if you could test every possible kind of input, then maybe you could prove that your code has no bugs. Dijkstra anticipated this objection as well, and proved that such exhaustive testing was physically impossible. Here's the quote from his original paper which dealt with the microcode contained in a CPU:

Let us restrict, for a moment, our attention to the hardware and let us wonder to what extent one can convince oneself of its being properly constructed. Some years ago a machine

was installed on the premises of my University; in its documentation it was stated that it contained, among many other things, circuitry for the fixed-point multiplication of two 27-bit integers. A legitimate question seems to be: "Is this multiplier correct, is it performing according to the specifications?".
The naive answer to this is: "Well, the number of different multiplications this multiplier is claimed to perform correctly is finite, viz. 254, so let us try them all." But, reasonable as this answer may seem, it is not, for although a single multiplication took only some tens of microseconds, the total time needed for this finite set of multiplications would add up to more than 10,000 years! We must conclude that exhaustive testing, even of a single component such as a multiplier, is entirely out of the question.

Today, you might write a function that takes two 32-bit ints. Using Djikstra's calculations, if you could test a million calculations a second, then testing all possible combinations of those two 32-bit ints would take you 585,000 years.



Slide 3

Questions About Testing

Duration: 00:02:40
Advance mode: Auto

Notes:

To meaningfully test your program, you have to start by asking three questions:

1. First, what kind of tests are needed? Since you can't test every single combination of input, you'll need to provide some kind of representative data. Specifically, you'll want a sampling of data that includes those values that are most likely to fail and expose the errors in your code.
2. Second, where are you going to get your test data. While informal tests can generate the test data using keyboard input, that quickly becomes very tedious. Normally you'll want to hand-select your input values and then write a representative selection of tests using those values. Sometimes, it may be useful to use a loop and random-number generation as well to generate a larger sample of input values.
3. Finally, how do you tell if your program works? You not only have to carefully decide what values to feed into your program, but for every input value, you absolutely must know exactly what the expected output should be. Simply calling a method is not the same as testing the method. Testing is when you call the method and then compare the expected output with the actual output. For that to work, you have to know the expected output ahead of time.

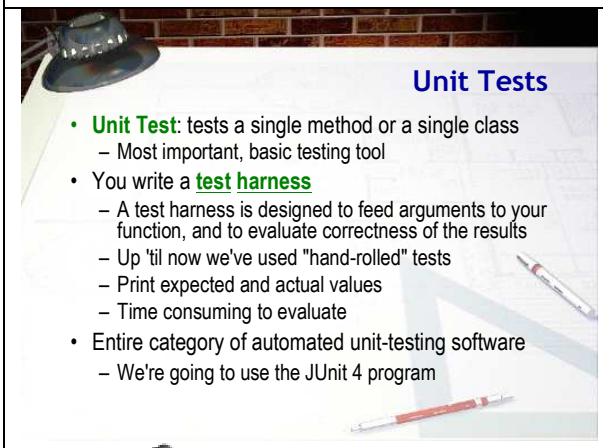
When you test your programs, there are really three kinds of techniques you can use to expose, locate and correct the defects in your code.

- Assertions are statements that are placed inside your code that make a method's preconditions and postconditions explicit. For instance, if a method takes a parameter and it assumes that the parameter always

contains the values 1, 2, 3, or 4, and assertion would check the parameter value before it was used, and then print an error message if it was incorrect. Assertions are used to expose errors when calling methods, not when writing them.

- Tracing or Logging is used when errors occur during program runtime, but you don't want to interrupt the program like you would with an assertion. Instead a log file is created that can later be examined to track down errors.
- Finally, debugging is interactive examination of your program. You'll learn how to debug your code using the Eclipse debugger in the Swatting Bugs lecture this week.

While assertions, logging and debugging are all important techniques, in this lecture, we want to look at the single most important testing tool at your disposal: the unit test.



Slide 4

Unit Tests

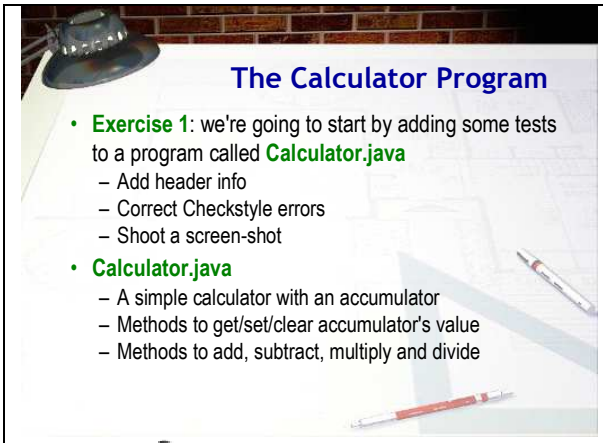
Duration: 00:01:22
Advance mode: Auto

Notes:

A unit test is sometimes called a test harness or just a testing program. A test harness is a program that is designed to feed arguments to your class or methods and then, to evaluate the correctness of the results. Your textbook uses "hand-rolled" unit tests, starting in Chapter 2, most of them very similar to the PongBallTest program we wrote when you were first learning to design your own classes. These hand-rolled tests call the methods in the class, and each time they call a method, the test prints out the expected and actual values of the object's instance variables.

Hand-rolled tests are fine, but the real problem comes with evaluating the results. When we wrote PongBallTest, for instance, many of you didn't notice if the actual values for x and y were different from the expected values because there was so much output. Carefully examining each line of output is very time consuming. That's where automated unit tests are much more efficient.

Automated unit testing software is an entire category of software. The most popular is the JUnit family of programs. The most recent version of JUnit is version 4, and that's what we'll use with our Eclipse programs.



Slide 5

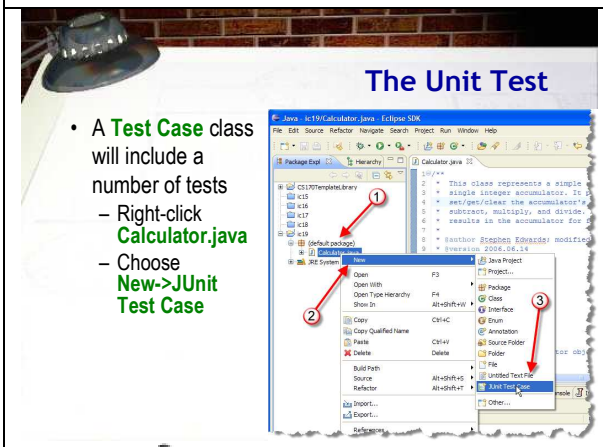
The Calculator Program

Duration: 00:01:14
Advance mode: Auto

Notes:

We're going to start by adding some tests to a program called Calculator.java. You'll find this inside your ic17 project. Go ahead and open it in your editor and add the header information. Note that there's at least one (possibly more) Checkstyle errors. If you haven't activated Checkstyle for your code, right-click the project, choose Properties and on the Checkstyle tab, choose the VTCS checks as the global checks to apply. Correct the Checkstyle errors and for Exercise 1 shoot a screen shot. Start a new section in your IC exercise document and paste the screenshot there.

Now, spend a few minutes looking through the Calculator class. Notice that the class has a single field which acts as the accumulator. There are methods to get, set and clear the accumulator, as well as the normal four functions that you'd expect in a calculator: add, subtract, multiply and divide. Each of these operations puts its result back into the accumulator and uses the accumulator as its left-hand operand.



Slide 6

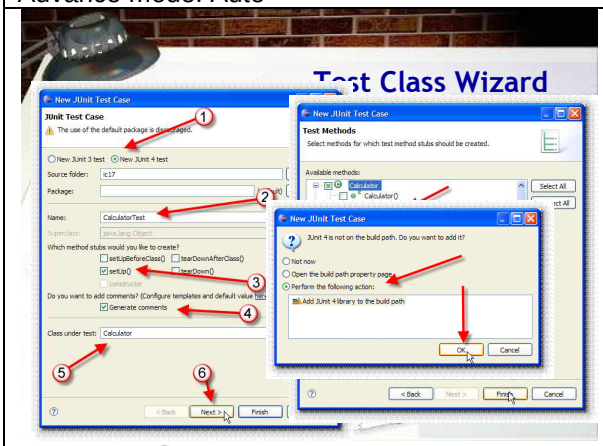
The Unit Test

Duration: 00:00:32
Advance mode: Auto

Notes:

With JUnit, you'll put your unit tests inside a Test Case class. The class can include a number of tests.

To create the CalculatorTest class Right-click on the Calculator.java file in the Package Explorer (on the left) and then select New->JUnit Test Case. This will activate a wizard in Eclipse to help you create your test case class for the Calculator class.



Slide 7

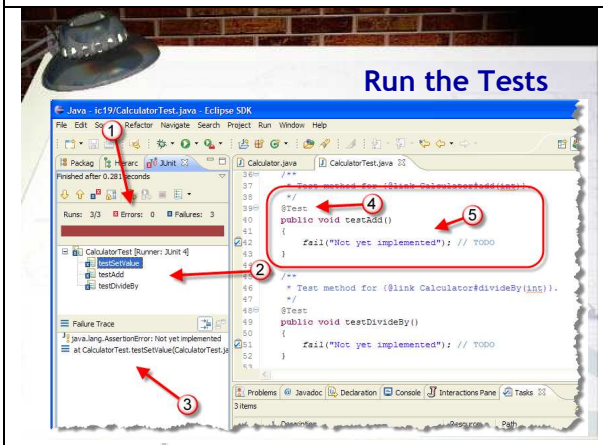
Test Class Wizard

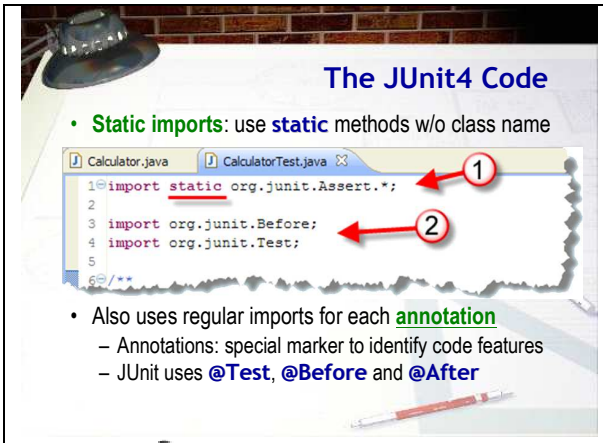
Duration: 00:01:32

Notes:

When the Test Class Wizard appears, start by clicking the radio button to select JUnit 4 as the testing framework (at # 1). For most of the other items, you can use the default values. Notice how the name CalculatorTest was automatically generated. You'll also want to make sure that the checkbox next to setUp() is selected to generate a stub for your test fixture. I'd go ahead and have Eclipse generate comments as well (that's # 4 in this slide.) Finally, make sure that Calculator is the class under test (#5) and then click the Next button.

In the next step of the wizard, you will see a tree view showing the methods provided by the Calculator class. For each method you check in this view, the wizard will create a test case stub. For this example, turn on setValue, add, and

Advance mode: Auto	divideBy (even though my slide only shows setVlaue and divideBy). This will generate empty test cases for these methods. When you're done, click the Finish button. If this is the first test case in this project, you'll be told that the JUnit libraries are not in the build path. Just click the radio button shown here and click OK, and Eclipse will add the necessary libraries. Then, Eclipse will generate a new CalculatorTest.java file in your project. It will also open this new file in the editor.
 **Slide 8** **Run the Tests** Duration: 00:02:02 Advance mode: Auto	**Notes:** Now select the CalculatorTest.java file in the Package Explorer view (the left pane). From the Run menu, select Run As->JUnit Test. This will run all of the test cases in your new test class (remember, the tests are just stubs at this point!). You can see the results in the JUnit tab to the right of to the Package Explorer tab. The test case stubs that were generated by the wizard have all failed. Let's look at the output briefly: 1. At the top of the window is a little tool bar you can use to re-run your tests. Right below that, at #1 is what I call a "scoreboard". It tells you how many tests were run, how many had errors and how many failed. (Errors mean that the test couldn't be run for some reason, like a missing class.) Below that is a bar that will contain green and red segments: red for failures and green for successes. In our case all the tests fail, so it's solid red. 2. At #2 is a tree showing each test that failed. By clicking on the test you can see a more detailed message in the Failure Trace pane, which I've marked #3 below. 3. Return to your CalculatorTest.java file in the editor and look at the code. You will see that each test case is a public void method taking no arguments. It is preceded by a Java annotation, @Test, that is used by JUnit to identify test methods. By convention, the test methods begin with the prefix "test"; this was required in JUnit 3 but is no longer required in JUnit 4. Each generated test case stub contains just a single line in its body: fail("Not yet implemented."); Which I've marked as #5. Now, you need to write the bodies for these test case methods.



Slide 9

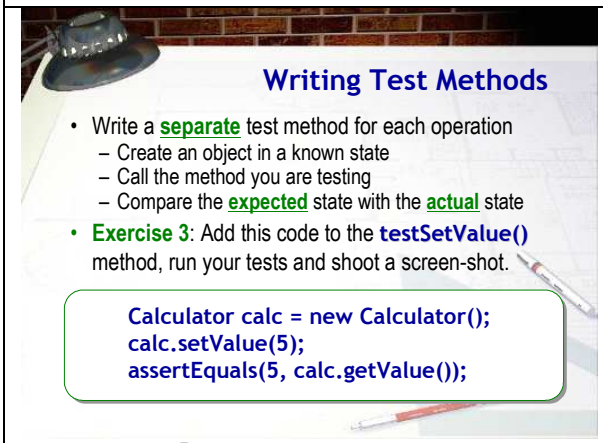
The JUnit4 Code

Duration: 00:01:42
Advance mode: Auto

Notes:

Before we write our tests, though, let's look a little more at the JUnit Test Case file and some of the Java features it uses.

- The first feature is the use of static imports. When you add a static import like that shown in line 1, you can then use any of the static methods in the imported class without using the class name. Notice that when you use a static import, the star at the end of the line means "import all of the static methods in the specified class". With a regular import, the star means "import all of the classes in the specified package". If you were to add import static java.lang.Math.*; to your code, for instance, you could then use methods like min() and max() without having to add the Math prefix.
- The second feature employed by JUnit 4 is the use of annotations. Annotations are special markers added to your code that allows an external tool (like JUnit) to process it in special ways. JUnit 4 uses the annotations @Test to identify test methods, @Before to identify methods that should be called before each test, and @After to identify methods that should be run after each test is complete. Each of these annotations needs to be imported using a regular (not static) import like that shown at #2 in my slide. Notice that I don't have an @After import since I didn't specify a tearDown() method when filling in the Test Case wizard.



Slide 10

Writing Test Methods

Duration: 00:01:12
Advance mode: Auto

Notes:

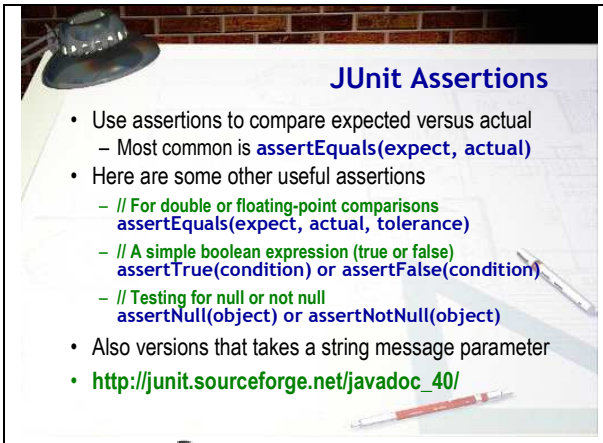
To test your class, you'll write at least one separate test method for each operation. Inside the test method (or inside the setUp() method, if you create one), you'll create an object in a known state, just like you did with BlueJ. Then, you'll call the different mutator methods that your are testing. After calling the mutators, you'll call an accessor method and compare the expected value with the actual value.

Let's try a simple example. Here's some code you can add to the testSetValue() method. Delete the line of code that the method currently contains and replace it with the code shown here. Then, run your tests and snap a screenshot of the JUnit Window.

Note how this method:

- Creates a Calculator object
- Modifies the calculator object by calling setValue
- Compares the expected value (5) with the actual value returned by calling getValue().

This last step is done by using a JUnit assertion. Let's go ahead and look at that now.



Slide 11

JUnit Assertions

Duration: 00:01:51
Advance mode: Auto

Notes:

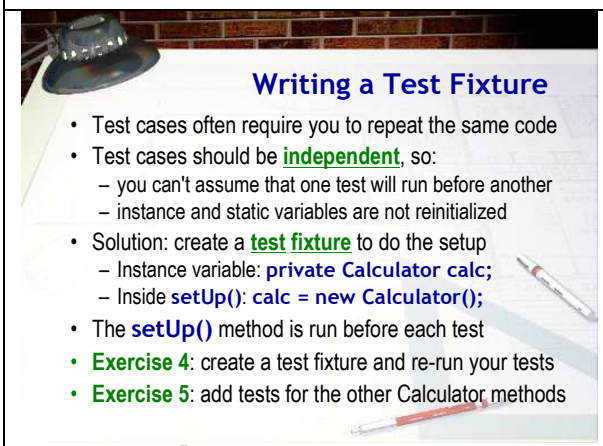
Assertions are special methods that JUnit uses to compare actual and expected values. The one you'll use most often is assertEquals(). When you use assertEquals(), the first parameter will be the value you expect and the second parameter will be the actual value produced by calling one of your object's accessors.

Here are some other useful assertion methods that you can use.

- For double or floating-point methods, you should normally use this version of assertEquals(). It works just like the first version, except you supply a tolerance value that is used to determine when two values are effectively equal. You can pass 1E-14 as a general-purpose tolerance value.
- For boolean methods or Boolean expressions, use the assertTrue() and assertFalse() methods. You can also use these in place of assertEquals() by creating an expression using relational operators, but if you do, then the error messages are not very informative. ("You failed the test. Too bad.").
- Finally, for objects, you sometimes need to test if an object reference is null or notNull, which you can do with these assertions.

For each of these assertions, you can supply an optional additional first String parameter containing an error message you want JUnit to display in the event that the test fails.

If you'd like to see more of the JUnit 4 assertions you can look them up in the online JavaDoc at http://junit.sourceforge.net/javadoc_40/.



Slide 12

Writing a Test Fixture

Duration: 00:02:10
Advance mode: Auto

Notes:

In the test method that we just wrote, we first created a Calculator object set to its default state. In fact, probably all of your methods will require this exact line of code.

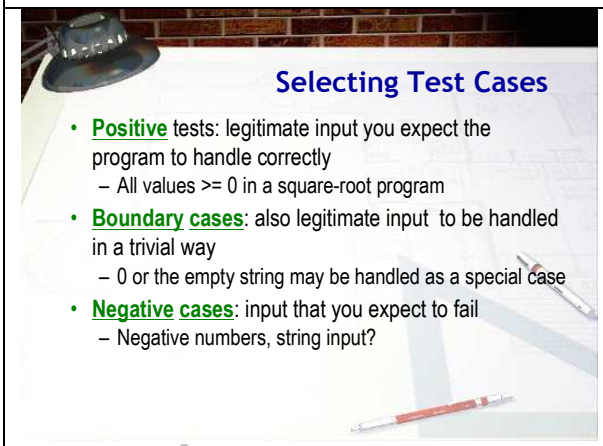
To simplify your code, you might be tempted to create the Calculator object as an instance variable. Unfortunately, that won't work, because you want all of your test methods to be completely independent. If you create the Calculator as an instance variable, it will be initialized when the class loads, but subsequent test methods won't be reinitialized; you won't be running with a clean slate. You also can't assume that test methods will run in any particular order, so you can't create sequences of tests that take prior method modifications into account. Basically every test has to be self-contained.

That doesn't mean that you have to duplicate the object creation in each test method though. The solution is to create an instance variable and then create a setUp() method that initializes the instance variable. This setUp() method, known

as a test fixture, will be run before each test, so each test starts with a clean slate.

Let's see if you can do that. For Exercise 4, create a test fixture and then modify the testSetValue() method by removing the line that creates the local variable named calc. Run the method and make sure that you still get a green light for this test method. (The other two will still fail, of course.).

One you have the test fixture done, let's finish up CalculatorTest by writing test methods for all of the other methods in Calculator. You can just copy the existing, working method (that's what I do), and then just change the name and modify the code. When you're done, for Exercise 5, show me a screenshot of all the tests passing in JUnit.



Slide 13

Selecting Test Cases

Duration: 00:01:25
Advance mode: Auto

Notes:

Now that you know how to write a test case, let's talk about what kind of test cases you should write. You already know that you should have at least one test case for each method in the class you are testing, but actually, you'll need quite a few more, each one handling a particular kind of input.

- First, you should have some values that represent legitimate values that your program is expected to handle normally. You don't need a lot of these. In a square root program, you'd test some values greater than 0. In a grade calculating program you'd test values for each of the grades: A – F.
- Next, you need to test boundary cases. Boundary cases include those values that are legitimate, but likely to produce incorrect output if there is an error in your code. If you are calculating letter grades from percentages, for example, testing 85% for the "B" grade would be a regular legitimate value. You'd also need to test the boundary conditions 80% and 79% in case you had used the relational operators incorrectly. These would be the boundary conditions.
- Finally, you need to handle some negative cases: input that you expect to fail. You want to make sure that if you supply the wrong kinds of values, that the method doesn't some how magically produce output.

Slide 14

Regression Testing

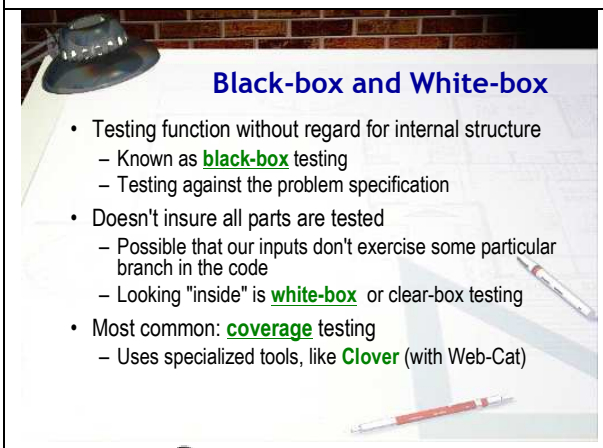
Duration: 00:01:00
Advance mode: Auto

Notes:

Whenever you fix a bug in your code, you want to make sure that the bug doesn't reappear later. Of course, this really applies to code in the "real" world, and not the code that you'll turn in for homework assignments. You don't want a bug that was fixed in Version 1 of your program to magically reappear when you release Version 5, a problem known as cycling.

The best way to prevent this is to add a test to your Test Case class that specifically tests for that bug. As time goes on, the Test Case class will get larger and larger as more bugs are fixed and tests are added.

Then, every time you fix a new bug, you'll run all of the tests you've collected, so your new fix can't reintroduce an old bug. This practice of testing your code against a collection of tests from past bugs is known as regression testing, and it's another reason to keep your test data.



Slide 15

Black-box and White-box

Duration: 00:01:10
Advance mode: Auto

Notes:

One last type of testing you should know about is the difference between black-box and white-box testing.

So far, all of the testing that we've done tests the program's actual behavior against the expected output. That means we're testing against what the program should be doing, or the specification. This is called black-box testing.

The only problem with black-box testing is that it doesn't ensure that all the internal portions of the code are actually tested. It's possible that your inputs don't test every single branch in an if or switch statement.

White-box testing "looks inside" your code and tries to make sure that every portion of your code is actually tested. The most common kind of white box testing is called coverage testing. Coverage testing uses specialized tools that are actually very expensive. The most common one is called Clover. While we don't have a copy of that here, Web-Cat actually uses Clover to measure what percentage of your code is actually tested.

